

CIS 5800, Machine Perception, Spring 2026
Homework 3C (Coding)
Due: Wednesday Mar. 26 2026, 11:59pm ET

Instructions

- This is the coding part for Homework 3 and worth **100 points**
- You need to check the released package containing **python files** and fill in all the sections where `##### STUDENT CODE START #####` is indicated.
- You can run the provided **hw3.ipynb** to debug, use **visualizer.py** to interactively adjust your 3D kitchen, and verify your implementation.
- Start early! Please post your questions on [Ed Discussion](#) (using the appropriate thread category) or come to office hours!

Submission

- For this coding part, you need to submit **6 Python files** (Please refer to to Sec. 1.1 for details) in total to the [Gradescope](#).

3D Reconstruction: From Pixels to the Franka Kitchen

Your TA Tony is busy training a new manipulation policy in the **Franka Kitchen** at GRASP Lab. While the robot was moving, he captured two frames from the ZED 2 camera. He wants to use these to build a 3D miniature model of his workspace to check for obstacles.

Using your freshly learned computer vision skills, you are tasked to recover the robot's camera motion and the 3D structure of the kitchen (as shown in Figure 2).

- You need to recover the 3D transformation (R, T) between the two views, such that $P_1, P_2 \in \mathbb{R}^3$ describe the same scene point in frame 1 and 2, and $\boxed{P_2 = RP_1 + T}$.



(a) Left camera view



(b) Right camera view

Figure 1: The ZED 2 stereo views from the DROID setup.

Camera Calibration Details: Tony has already calibrated the ZED 2 camera for you. Both views share the same intrinsic matrix K :

- **Intrinsics:** $K = \begin{pmatrix} f & 0 & u_0 \\ 0 & f & v_0 \\ 0 & 0 & 1 \end{pmatrix}$ with $\mathbf{f} = \mathbf{967}$ and principal point $(\mathbf{u}_0, \mathbf{v}_0) = (\mathbf{640}, \mathbf{360})$.
- **Note:** These images are **rectified**. You can assume zero radial distortion for this assignment.

We will use the following notations:

- The notation $\mathbf{x} = (x, y, 1)$ (homogeneous metric coordinates) will be used for calibrated projections:

$$\boxed{P_1 = \lambda_1 \mathbf{x}_1, \quad P_2 = \lambda_2 \mathbf{x}_2}$$

- The notation $\mathbf{u} = (u, v, 1)$ (homogeneous pixel coordinates) will be used for uncalibrated projections:

$$\boxed{\mathbf{u}_1 \sim K \mathbf{x}_1, \quad \mathbf{u}_2 \sim K \mathbf{x}_2}$$

$$\boxed{\mathbf{u} \sim K \mathbf{x} \Leftrightarrow u = fx + u_0, \quad v = fy + v_0}$$

- $\text{epi}(\mathbf{x}_1)$ is the epipolar line in camera 2 corresponding to $\mathbf{x}_1$.

1.1 What to submit

Submit to Gradescope only the following files:

- `lse.py`
- `plot_epi.py`

- `pose.py`
- `ransac.py`
- `recon3d.py`
- `show_reproj.py`

1.2 Estimation of the Essential Matrix (54 pts)

You will begin by reconstructing the kitchen scene from SIFT matchings between the two ZED 2 views (Figure 2). While the robot’s metal surfaces and the kitchen’s reflective tiling can make matching difficult, the SIFT descriptors have been pre-computed for you using OpenCV.

Before diving in, we recommend reading the OpenCV tutorials on [Feature Detection and Description](#) and ensuring you understand the transition from pixel coordinates $\mathbf{u}$ to calibrated coordinates $\mathbf{x}$ using your ZED 2 intrinsics.

Also, quickly read the whole script to understand the overall pipeline.

1. **Least-squares estimation** Complete the function `least_squares_estimation` in `lse.py` so that it takes two $N \times 3$ matrices of matched, calibrated points $\mathbf{X}_1, \mathbf{X}_2$ as input and estimates E using the SVD decomposition method as in the 8-point algorithm described in the lecture notes and the “E-matrix” handout. Don’t forget to project the E you obtain onto the space of essential matrices by redecomposing `[U,S,Vt] = np.svd(E)` and returning $U \text{diag}(1, 1, 0) V^\top$.
2. **RANSAC estimation** The estimation of E can be made much more robust by selecting sets of points that reach a common agreement: in this section you will implement a basic RANSAC algorithm to eliminate outliers (spurious matchings) and obtain a better estimate of E .
 - (a) One iteration of the algorithm uses the code from the previous question to estimate the essential matrix E based on a random set of 8 correspondences, and then evaluates how good E is for the other pairs $(\mathbf{x}_1, \mathbf{x}_2)$. If E is perfect, for all pairs $(\mathbf{x}_1, \mathbf{x}_2)$, $\mathbf{x}_2$ lies exactly on the epipolar $\text{epi}(\mathbf{x}_1)$ computed from E and $\mathbf{x}_1$. Therefore, we will use the distances to the epipolars as an error measure. The distance of a matching point to the epipolar is the following:

$$d(\mathbf{x}_2, \text{epi}(\mathbf{x}_1))^2 = \frac{(\mathbf{x}_2^T E \mathbf{x}_1)^2}{\|\hat{\mathbf{e}}_3 E \mathbf{x}_1\|^2}.$$

where $\mathbf{e}_3 = (0, 0, 1)^T$ and $\hat{\mathbf{e}}_3$ is the skew-symmetric matrix of $\mathbf{e}_3$.

- (b) Complete the function `ransac_estimator` in `ransac.py` that takes a set of matching points and estimates E using RANSAC. The algorithm steps are the following:

- Pick a random set of 8 pairs, estimate E using them and compute the individual residuals for all the other pairs $(\mathbf{x}_1, \mathbf{x}_2)$: $d(\mathbf{x}_2, \text{epi}(\mathbf{x}_1))^2 + d(\mathbf{x}_1, \text{epi}(\mathbf{x}_2))^2$.
- Count how many residuals are lower than $\varepsilon = 10^{-4}$ (consensus set), and if this count is the largest so far, store the current estimate of E as the best estimate so far,
- Iterate as many times as needed according to the probability of failure

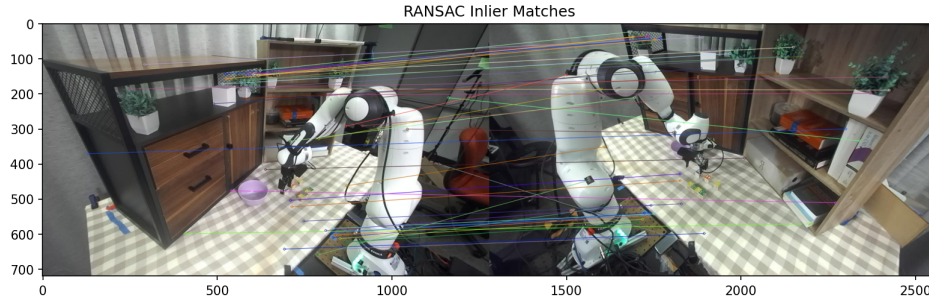


Figure 2: RANSAC inlier matches between the left and right ZED 2 views of the Franka Kitchen. Each colored line connects a pair of corresponding keypoints that survived the RANSAC outlier rejection step.

3. **Drawing the epipolar lines** You can now use the essential matrix to plot epipolar lines in the images, as shown in figure 3. Note that E defines epipolars for *calibrated* points $(\mathbf{x}_1, \mathbf{x}_2)$. For the uncalibrated points, we showed in class there is a perfectly identical relationship by substituting the **fundamental matrix** F to E :

$$\mathbf{x}_2^T E \mathbf{x}_1 = \mathbf{u}_2^T \underbrace{K^{-T} E K^{-1}}_F \mathbf{u}_1 = 0 \Leftrightarrow \mathbf{u}_1 \in \text{epi}_F(\mathbf{u}_2)$$

Complete the function `plot_epipolar_lines` in `plot_epi.py` that draws the epipolar lines for a set of matching image points (uncalibrated pixel coordinates). Note that you just need to fill in the equations, the “drawing” part is already done for you. If your RANSAC and K matrix are correct, the epipolar lines in the second image should pass precisely through the matching SIFT points on the Franka arm (see Figure 3).

1.3 Pose recovery and 3D reconstruction (54 pts)

It is now time to recover the transformation R, T between the two cameras. For a given estimate of E , remember that there are two possible solutions for $(\hat{T}, R)$ (twisted pair

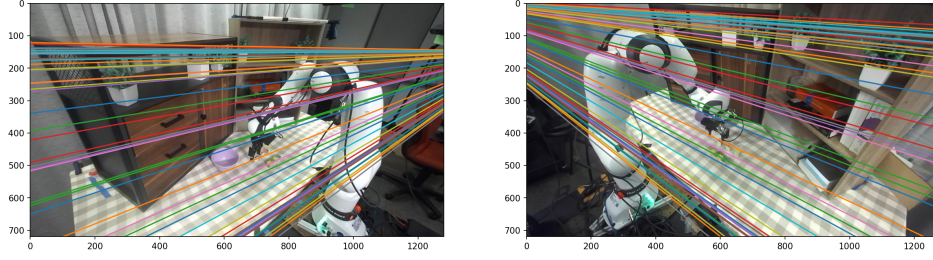


Figure 3: Epipolar lines in the Franka Kitchen. Each line represents the possible location in Image 2 of a point observed in Image 1.

ambiguity):

$$\begin{aligned}(\hat{T}_1, R_1) &= (UR_Z(\frac{\pi}{2})\Sigma U^T, UR_Z(\frac{\pi}{2})^T V^T) \\ (\hat{T}_2, R_2) &= (UR_Z(-\frac{\pi}{2})\Sigma U^T, UR_Z(-\frac{\pi}{2})^T V^T)\end{aligned}$$

where (U, Σ, V) is the SVD decomposition of E and $R_Z(\frac{\pi}{2}) = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$.

1. The third column of U has associated hat matrix $UR_Z(+\frac{\pi}{2})U'$. Also the opposite of the third column of U has associated hat matrix $UR_Z(-\frac{\pi}{2})U'$. Therefore, in the two solutions above, you can output T instead of its hat matrix and simply use the third column of U and its opposite for T_1 and T_2 .
2. The sign switch between $R_Z(+\frac{\pi}{2})$ and $R_Z(-\frac{\pi}{2})$ inside the decompositions of the solutions is equivalent to a left-multiplication of the solutions by $R_T(\pi)$ where T is the third column of U .
3. Complete the function `pose_candidates_from_e` in `pose.py` that takes E as an input and returns four possible solutions for (R, T) :
 - the twisted pair for E ,
 - *and* the twisted pair for $-E$. This is because if we find E that satisfies all the epipolar constraints $\mathbf{x}_1^T E \mathbf{x}_2 = 0$ (part 1.2), the opposite matrix $-E$ also verifies the constraint since the right-hand side is zero, and we don't know a priori whether E or $-E$ is the essential matrix we are looking for.

ADDENDUM NOTE: Only perform SVD on the single, given E and denote the result U, Σ, V^T . Then, use the decomposition in the following order (ordering is purely enforced for autograding purposes):

- (a) T from U , $R = UR_z(\frac{\pi}{2})^T V^T$
- (b) T from U , $R = UR_z(-\frac{\pi}{2})^T V^T$
- (c) T from $-U$, $R = UR_z(\frac{\pi}{2})^T V^T$
- (d) T from $-U$, $R = UR_z(-\frac{\pi}{2})^T V^T$

4. **Triangulation** Given a candidate (R_i, T_i) , we can now attempt to reconstruct all the points from the pairs (x_1, x_2) , and check whether (R_i, T_i) is the correct transformation (we need to pick one out of the four candidates): we will pick the candidate (R_i, T_i) that has the highest number of reconstructed points *in front of both cameras*. Consider one of the matchings (x_1, x_2) : we can reconstruct the 3D point by computing the intersection of the two rays coming from camera 1 and 2:

$$\lambda_2 \mathbf{x}_2 = \lambda_1 R \mathbf{x}_1 + T,$$

In practice, the equality doesn't hold (the two rays don't intersect perfectly) and you need to do a least square estimation of λ_1, λ_2 .

For a given pair $(\mathbf{x}_1, \mathbf{x}_2)$, formulate the linear system such that $c\lambda_1\lambda_2$ is its solution. Complete the part of `reconstruct3D` in `recon3d.py` that loops through the four transformation candidates (T, R) and all pairs $(\mathbf{x}_1, \mathbf{x}_2)$, and solves for the depths λ_1, λ_2 .

The selection of the right (T, R) out of the four possibilities is already coded for you but you should take a look at it and make sure you understand it: we simply count for each candidate (T_i, R_i) how many points are in front of both cameras (namely both $\lambda_1, \lambda_2 > 0$) and keep the transformation that achieves the maximum number.

5. You can now run the whole pipeline in the notebook to estimate the transformation (T, R) between the two cameras and the relative positions of the 3D points. Figure 4 shows an example of output from `plot_reconstruction`, shown in the (X, Z) plane (we're on top of the scene, looking down at it like a street map). Note that all depths and the translation (baseline) are up to a scale.
6. Complete the function `show_reprojections` in `show_reproj.py` to compare image points in a given camera to the reprojection of image points from the other camera. All you need to do in this function is apply the camera projection model (and be careful about the expression of the 3D transform from 1 to 2 and from 2 to 1).

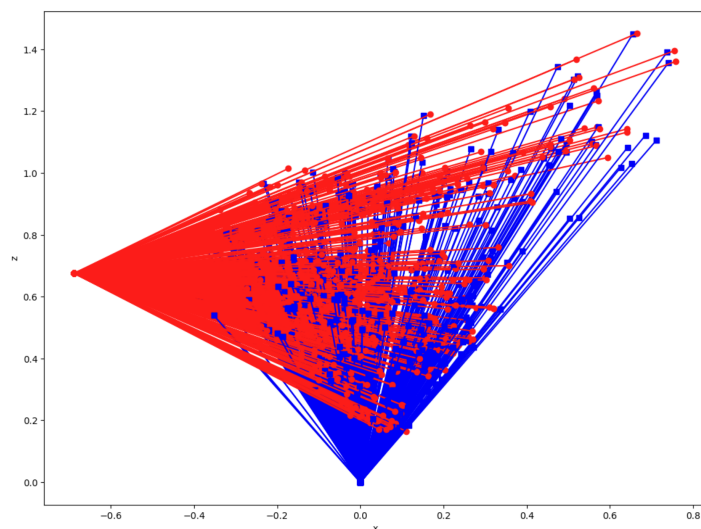


Figure 4: Top-down view of the scene, origin at camera 2. Red points (circles) and their corresponding rays from Camera 1; blue points (squares) from Camera 2.

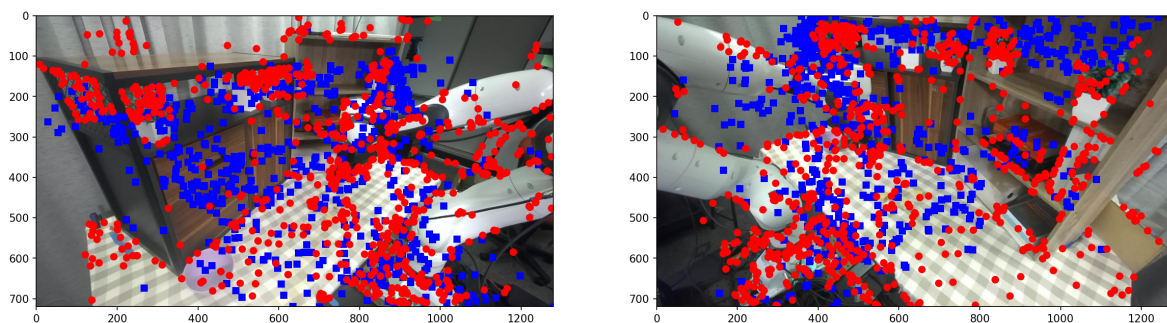


Figure 5: You should see reprojection on the two images when you finished.

2 Interactive Visualizer

To help you build intuition for how each parameter affects the reconstruction pipeline, we provide an **interactive 3D Reconstruction Visualizer** alongside the notebook (Figure 6). Adjusting any slider immediately reruns the full pipeline—from feature matching through RANSAC to triangulation—so you can observe in real time how choices like the Sampson distance threshold ε or the number of RANSAC iterations change the set of inliers, the epipolar geometry, and the final 3D scatter plot. The left panel exposes all algorithm parameters; the right panel updates the four linked views: feature matches, epipolar lines (left and right), 3D reconstruction, and reprojections.

We encourage you to try with extreme values—a very loose ε or very few iterations—before settling on the parameters you report in your writeup.

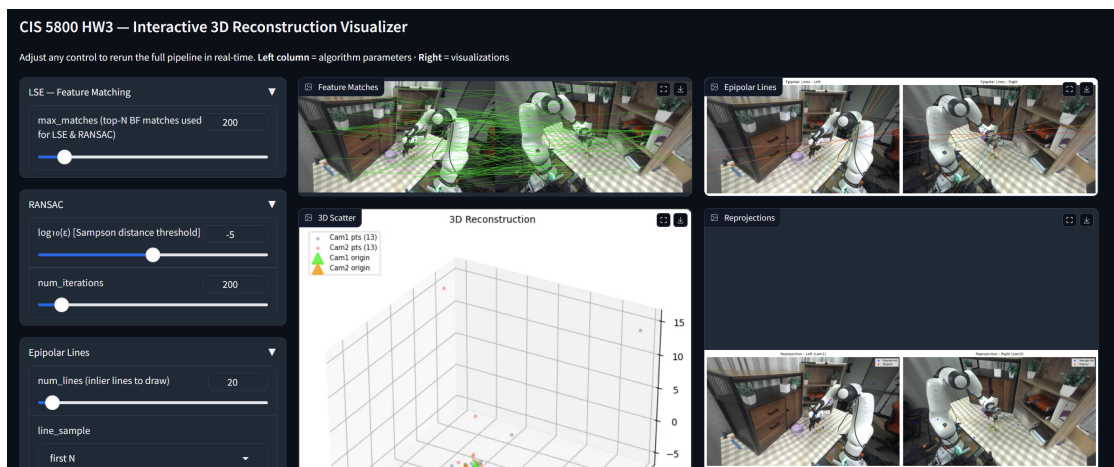


Figure 6: Open <http://localhost:7860> in your browser to visualize it

3 Optional: Neural Geometry with VGGT

Notice, this part is optional and not counted into Gradescope, but we encourage you to try it out!

Classical SfM pipelines rely on hand-crafted features (SIFT) and algebraic solvers (the 8-point algorithm), which can struggle with challenging lighting conditions or reflective surfaces.

VGGT (Visual Geometry Grounded Transformer) is a recent foundation model from Meta that predicts 3D geometry directly from image patches. Rather than extracting keypoints explicitly, VGGT learns epipolar constraints into its network, enabling robust pose and correspondence estimation even in low-texture regions where geometry-based methods tend to fail.

1. Classical vs. Learned Features

A helper script `vggt_inference.py` is provided to run a pre-trained VGGT model the given Franka Kitchen images and output the estimated pose (R, T). Run the script and visually compare the resulting point cloud against your RANSAC-based reconstruction.

VGGT works on *any* scene captured as a short video or a sequence of consecutive frames from the same camera. Simply shoot a short walkthrough of your room, desk, or any in-door environment, extract the frames, and try it yourself at the official demo:

<https://huggingface.co/spaces/facebook/vggt>

The only requirement is that frames come from the same camera model and have reasonable overlap between consecutive views.

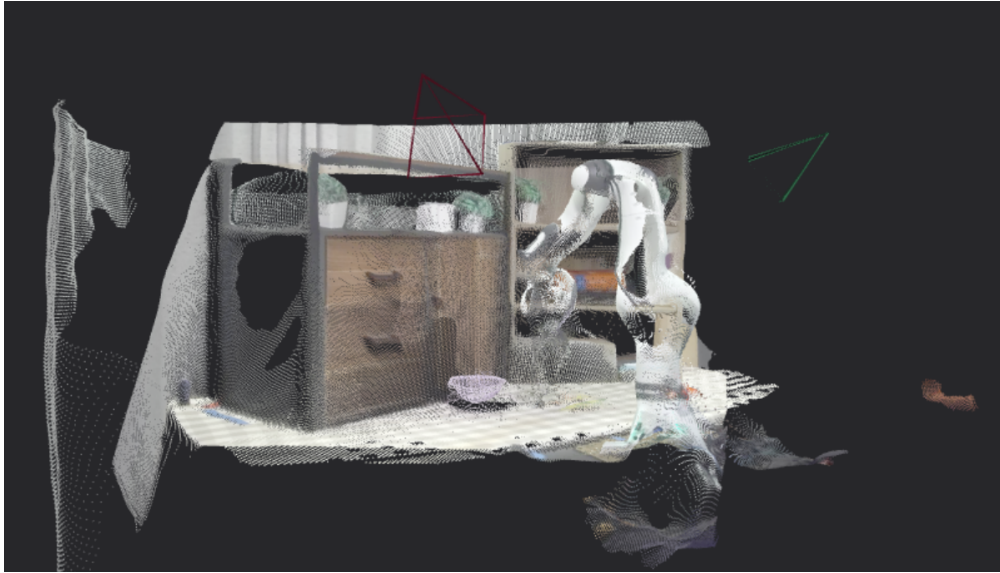


Figure 7: VGGT reconstruction of the Franka Kitchen scene. Colored frustums (red and green triangles) represent the two estimated camera poses;

2. Show Off Your Scene!

Export a screenshot of your best reconstruction and upload it to Gradescope. We also encourage you to share it on [Ed Discussion](#) for your most accurate (or most impressive) reconstructions!